FastAPI Notes

General Concepts x MEDSCAN.AI Project Usage

A side-by-side reference guide

1. What Is FastAPI?

FastAPI is a modern Python web framework for building HTTP APIs. It uses Python type hints to auto-validate request/response data and auto-generates interactive docs (Swagger UI at /docs, ReDoc at /redoc). It is built on top of Starlette (ASGI) and Pydantic.

General FastAPI Concept	MEDSCAN.AI Usage
A web framework that maps URL paths to Python functions called 'route handlers'.	MEDSCAN creates the FastAPI app in app/main.py: app = FastAPI(title='MEDSCAN.AI')
Every route handler returns data; FastAPI serializes it to JSON automatically.	Routes like GET /api/reports return Python dicts/Pydantic models -> JSON.
Auto-generates /docs (Swagger UI) so you can test endpoints in the browser.	Developers test /api/upload, /api/status/{id} directly in the browser during dev.
Uses ASGI (async), so it handles many simultaneous requests efficiently.	Concurrent report uploads and status polls are handled without blocking.

Concept: minimal app

```
# Minimal FastAPI application
from fastapi import FastAPI
app = FastAPI()

@app.get('/hello')
def hello():
    return {'message': 'Hello, world'}
```

MEDSCAN.AI: app/main.py

```
# MEDSCAN.AI -- app/main.py
from fastapi import FastAPI
from app.routers import auth, reports, ml

app = FastAPI(
    title='MEDSCAN.AI',
    description='Medical report risk analysis API',
    version='1.0.0',
)
app.include_router(auth.router,    prefix='/auth')
app.include_router(reports.router, prefix='/api')
app.include_router(ml.router,      prefix='/api')
```

2. Routers -- Splitting Routes Across Files

As an API grows, keeping every route in main.py becomes unmanageable. FastAPI's APIRouter lets you define routes in separate files and then mount them all into the main app with include_router(). Each router can have its own prefix, tags, and dependencies.

General FastAPI Concept	MEDSCAN.AI Usage
APIRouter() creates a mini-app that can hold its own routes.	MEDSCAN has routers: auth.py, reports.py, ml.py, compare.py.
prefix='/auth' means all routes inside become /auth/login, /auth/register etc.	auth.router has prefix='/auth', so POST /auth/login, POST /auth/register.
tags=['Auth'] groups the routes under a named section in /docs.	reports.router uses tags=['Reports'] so docs are neatly grouped.
dependencies=[Depends(x)] applies x to EVERY route in the router.	reports.router passes Depends(get_current_active_user) to protect all report routes.

MEDSCAN.AI: routers/reports.py

```
# app/routers/reports.py
from fastapi import APIRouter, Depends
from app.dependencies import get_current_active_user

router = APIRouter(
    prefix='/api',
    tags=['Reports'],
    dependencies=[Depends(get_current_active_user)],
)

@router.get('/reports')
def list_reports(...):
    ...
```

3. Path Parameters and Query Parameters

Path parameters are part of the URL itself: /reports/{report_id}. Query parameters follow a '?' in the URL: /reports?limit=10&offset=0. FastAPI reads Python function arguments and maps them automatically -- if the name matches a path variable it is a path param; otherwise a query param.

General FastAPI Concept	MEDSCAN.AI Usage
{report_id} in the path string -> int report_id arg -> FastAPI validates it is an int.	GET /api/status/{report_id} -> def get_status(report_id: int, db: Session).
Query params are optional if given a default value (limit: int = 10).	GET /api/reports can accept ?limit=20 to control how many reports are returned.
FastAPI returns HTTP 422 automatically if a param fails type validation.	Sending /api/status/abc returns 422 Unprocessable Entity with a clear error message.

Concept: params

```
# Concept: path param + query param
@app.get('/items/{item_id}')
def get_item(item_id: int, q: str = None):
    return {'id': item_id, 'q': q}

# URL: /items/42?q=search
# -> item_id=42, q='search'
```

MEDSCAN.AI: GET /api/status/{report_id}

```
# MEDSCAN.AI -- routers/ml.py
@router.get('/status/{report_id}')
def get_status(
    report_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_active_user),
):
    report = db.query(Report).filter(
        Report.id == report_id,
        Report.user_id == current_user.id,
    ).first()
    if not report:
        raise HTTPException(status_code=404)
    return report
```

4. Request Body and Pydantic Schemas

When a client sends JSON in the request body (POST/PUT), FastAPI reads it and validates it against a Pydantic model. If validation fails, FastAPI returns HTTP 422 with a detailed error. Pydantic models also serve as response_model to filter what gets sent back to the client.

General FastAPI Concept	MEDSCAN.AI Usage
class LoginRequest(BaseModel): email: str; password: str	AuthRequest model receives email + password for POST /auth/login.
response_model=TokenResponse tells FastAPI to filter output through that schema.	POST /auth/login returns {access_token, refresh_token, token_type} -- nothing else.
Field(...) = required; Field(None) or Optional[str] = optional.	ReportResponse has Optional fields for result/extracted_metrics (null while processing).
Validators (@field_validator) run before the route handler is called.	Email fields are validated for correct format before reaching auth logic.

Concept: Pydantic models

```
# Concept: Pydantic request + response model
from pydantic import BaseModel

class LoginRequest(BaseModel):
    email: str
    password: str

class TokenResponse(BaseModel):
    access_token: str
    token_type: str

@app.post('/login', response_model=TokenResponse)
def login(body: LoginRequest):
    # FastAPI auto-parses JSON body -> LoginRequest
    ...
```

5. Dependency Injection with Depends()

Depends() is FastAPI's built-in dependency injection system. You write a function that returns something (a DB session, a user object, etc.) and declare it as a default argument using Depends(). FastAPI calls the dependency function automatically before calling your route handler. Dependencies can call other dependencies, forming a chain.

General FastAPI Concept	MEDSCAN.AI Usage
Depends(get_db) -> FastAPI calls get_db(), injects the Session into the route.	Every DB-touching route in MEDSCAN uses db: Session = Depends(get_db).
get_db() is a generator (yield) so cleanup (session.close()) runs after the response.	get_db() opens a SessionLocal, yields it, then closes in a finally block.
Dependencies chain: get_current_user calls Depends(oauth2_scheme) internally.	get_current_active_user -> get_current_user -> oauth2_scheme -> Authorization header.
Depends() on a router-level applies the dep to every route in that router.	All /api/reports routes are protected because reports.router uses Depends(get_current_active_user).

MEDSCAN.AI: full dependency chain

```
# MEDSCAN.AI dependency chain

# Step 1: database session
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# Step 2: extract JWT from Authorization header
oauth2_scheme = OAuth2PasswordBearer(tokenUrl='/auth/login')

# Step 3: decode JWT -> user object
def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: Session = Depends(get_db),
):
    payload = decode_jwt(token)  # raises 401 if invalid
    return db.query(User).get(payload['sub'])

# Step 4: check user is active
def get_current_active_user(
    user: User = Depends(get_current_user),
):
    if not user.is_active:
        raise HTTPException(403)
    return user
```

6. File Uploads with UploadFile

FastAPI handles multipart/form-data uploads through the UploadFile class. The client sends a form-data POST with the file attached. FastAPI reads the file into memory (or streams it) and exposes it as an UploadFile object. You can read the bytes, check the filename and content type, and then save or process the file.

General FastAPI Concept	MEDSCAN.AI Usage
File(...) declares the parameter as a required upload field.	POST /api/upload uses file: UploadFile = File(...).
UploadFile.read() returns bytes; UploadFile.filename gives the original name.	MEDSCAN reads the bytes and uploads them to Supabase S3 via boto3.
Content-Type header must NOT be set manually; the browser sets it with the boundary.	Frontend api.js note: do not set Content-Type for FormData uploads.
File validation (size, MIME type) must be done manually inside the route.	MEDSCAN checks file.content_type in ['image/jpeg','image/png','application/pdf'].

MEDSCAN.AI: POST /api/upload

```
# MEDSCAN.AI -- routers/reports.py
from fastapi import UploadFile, File

@router.post('/upload')
async def upload_report(
    file: UploadFile = File(...),
    report_type: str = Form(...),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_active_user),
):
    contents = await file.read()  # bytes
    # upload to Supabase S3
    s3 = _get_s3()
    s3.put_object(Bucket=BUCKET, Key=filename, Body=contents)
    # create DB record
    report = Report(user_id=current_user.id, filename=filename, ...)
    db.add(report); db.commit()
    # dispatch Celery task
    process_medical_report.delay(report.id, filename, report_type)
    return {'report_id': report.id, 'status': 'processing'}
```

7. HTTPException and Error Responses

Raise HTTPException to return an error response immediately and stop executing the route handler. FastAPI converts it to a JSON response with the status_code and a {detail: ...} body. You can also add custom headers (e.g. WWW-Authenticate for 401 errors).

General FastAPI Concept	MEDSCAN.AI Usage
raise HTTPException(status_code=404, detail='Not found') -> {detail: 'Not found'}	Report not found or not owned by user -> 404 with detail message.
raise HTTPException(401, headers={'WWW-Authenticate': 'Bearer'})	get_current_user raises 401 when token is missing, expired, or tampered.
422 Unprocessable Entity is raised automatically by FastAPI on Pydantic failures.	Sending wrong JSON to POST /auth/register returns 422 with field-level errors.
You can register @app.exception_handler(exc_type) for global custom handling.	MEDSCAN uses FastAPI's default handlers; no custom exception handlers yet.

Concept: HTTPException

```
# Concept: HTTPException usage
from fastapi import HTTPException

@app.get('/reports/{id}')
def get_report(id: int, db: Session = Depends(get_db)):
    report = db.query(Report).get(id)
    if not report:
        raise HTTPException(status_code=404, detail='Report not found')
    return report

# Response body: {"detail": "Report not found"}
# HTTP status: 404
```

8. JWT Authentication -- Login and Token Flow

JWT (JSON Web Token) is a signed string that proves who the caller is. The server creates it at login and signs it with a secret key. On every subsequent request the client sends the JWT in the Authorization: Bearer <token> header. FastAPI reads and verifies it to identify the user -- no database lookup needed for every request.

General FastAPI Concept	MEDSCAN.AI Usage
Login: POST /auth/login -> server creates and returns access_token + refresh_token.	POST /auth/login returns {access_token (30 min), refresh_token (7 days)}.
access_token is short-lived (minutes). Stored in localStorage on the frontend.	Frontend stores access_token in localStorage; attaches it to every API call.
refresh_token is long-lived (days). Used to get a new access_token silently.	POST /auth/refresh -> new access_token. apiFetch() retries automatically on 401.
OAuth2PasswordBearer extracts the Bearer token from the Authorization header.	oauth2_scheme = OAuth2PasswordBearer(tokenUrl='/auth/login') used in get_current_user.

MEDSCAN.AI: JWT flow

```
# MEDSCAN.AI -- auth flow

# On login:
def create_tokens(user_id: int):
    access  = jwt.encode({'sub': str(user_id), 'exp': now + 30min}, SECRET)
    refresh = jwt.encode({'sub': str(user_id), 'exp': now + 7days}, SECRET)
    return access, refresh

# On every protected request:
def get_current_user(token = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(token, SECRET, algorithms=['HS256'])
        user_id = payload['sub']
    except jwt.ExpiredSignatureError:
        raise HTTPException(401, 'Token expired')
    return db.query(User).get(user_id)

# Frontend silent refresh (api.js):
# if response.status == 401:
#   POST /auth/refresh -> new access_token -> retry request
```

9. Background Task Processing with Celery

OCR and ML inference take several seconds -- too long to block an HTTP response. FastAPI's built-in BackgroundTasks is simple but runs in the same process. MEDSCAN uses Celery + Redis instead: the route handler dispatches a task and returns immediately; a separate worker process runs the task in the background. The client polls GET /api/status/{report_id} until status changes to 'completed'.

General FastAPI Concept	MEDSCAN.AI Usage
process_medical_report.delay(id, filename, type) dispatches and returns a task_id.	POST /api/upload calls .delay() and immediately returns {report_id, status: processing}.
Celery worker picks up the task from Redis (the broker queue).	One Celery worker process runs with concurrency=1 to avoid GPU memory conflicts.
Task result is stored in Redis for 1 hour (result_expires=3600).	Status polls check the DB record (not Celery result) for report.status field.
@shared_task(bind=True) gives the task access to self.retry() for retries.	compare_reports retries up to 3 times with exponential back-off on ML service errors.

MEDSCAN.AI: Celery task dispatch

```
# MEDSCAN.AI -- task_queue/tasks.py

@shared_task(bind=True, max_retries=3)
def process_medical_report(self, report_id, filename, report_type):
    try:
        # 1. download file from S3
        # 2. POST file to ML microservice
        # 3. save OCR results to DB
        # 4. save prediction to DB
        # 5. mark report.status = 'completed'
        pass
    except Exception as exc:
        raise self.retry(exc=exc, countdown=2 ** self.request.retries)

# Route handler:
@router.post('/upload')
def upload(...):
    process_medical_report.delay(report.id, filename, report_type)
    return {'report_id': report.id, 'status': 'processing'}
```

10. Database Integration with SQLAlchemy

FastAPI has no built-in ORM, but SQLAlchemy (the most popular Python ORM) integrates naturally via the Depends() pattern. A SessionLocal factory creates one DB connection per request. The get_db() generator yields the session to the route and closes it in a finally block after the response.

General FastAPI Concept	MEDSCAN.AI Usage
create_engine(DATABASE_URL) creates the connection pool to PostgreSQL.	MEDSCAN connects to Neon PostgreSQL using DATABASE_URL from environment.
SessionLocal = sessionmaker(bind=engine) is the session factory.	get_db() creates a SessionLocal, yields it, closes it when done.
Base = declarative_base() is the parent class for all model classes.	User, Report, Prediction, Comparison all extend Base.
db.query(Model).filter(...).first() is how you query a single row.	get_status uses db.query(Report).filter(Report.id==id, Report.user_id==uid).first().

MEDSCAN.AI: database.py

```
# MEDSCAN.AI -- app/database.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

engine = create_engine(DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

11. CORS -- Allowing the React Frontend to Call the API

Browsers block cross-origin requests by default (CORS policy). Because the React frontend is served from a different origin than the FastAPI backend (different domain or port), CORS headers must be added to every response. FastAPI's CORSMiddleware handles this automatically.

General FastAPI Concept	MEDSCAN.AI Usage
add_middleware(CORSMiddleware, allow_origins=[...]) adds CORS headers to responses.	MEDSCAN allows the Vercel frontend URL and localhost:5173 (Vite dev server).
allow_credentials=True is required when the client sends cookies or Authorization headers.	Frontend sends Authorization: Bearer token, so credentials=True is needed.
allow_methods=['*'] permits GET, POST, PUT, DELETE, OPTIONS, etc.	MEDSCAN uses GET, POST, DELETE so ['*'] covers all of them.
The browser sends a preflight OPTIONS request first; CORSMiddleware handles it.	No special handling needed -- middleware responds to OPTIONS automatically.

MEDSCAN.AI: CORS setup

```
# MEDSCAN.AI -- app/main.py
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        'https://medscan-ai.vercel.app',
        'http://localhost:5173',
    ],
    allow_credentials=True,
    allow_methods=['*'],
    allow_headers=['*'],
)
```

12. Response Models and HTTP Status Codes

By default FastAPI returns HTTP 200 and whatever the route function returns. You can override both with decorator arguments. response_model filters which fields are sent to the client (useful for hiding internal fields). status_code sets the HTTP status code for successful responses.

General FastAPI Concept	MEDSCAN.AI Usage
@app.post('/users', status_code=201) returns 201 Created on success.	POST /auth/register returns 201 when a new user account is created.
response_model=UserOut strips password_hash and other internal fields.	UserOut schema exposes id, email, full_name -- not password_hash.
response_model_exclude_none=True removes null fields from the JSON output.	ReportResponse uses this so null result/metrics are omitted while processing.
Return Response(status_code=204) for endpoints that return no body.	DELETE /api/reports/{id} returns 204 No Content after deletion.

Concept: response model

```
# Concept: status code + response model
@app.post('/users', status_code=201, response_model=UserOut)
def create_user(body: UserCreate, db: Session = Depends(get_db)):
    user = User(email=body.email, ...)
    db.add(user); db.commit()
    return user  # FastAPI filters through UserOut schema

# Only fields in UserOut are sent -- password_hash is excluded
```

13. Environment Variables and Settings Management

Hard-coding secrets (DB passwords, JWT secret keys, API keys) in source code is a security risk. FastAPI projects typically use pydantic-settings or python-dotenv to load config from environment variables or a .env file. The settings object is created once and injected via Depends().

General FastAPI Concept	MEDSCAN.AI Usage
class Settings(BaseSettings): DATABASE_URL: str reads from env automatically.	MEDSCAN loads DATABASE_URL, SECRET_KEY, REDIS_URL, S3 keys from environment.
pydantic-settings validates types and raises on startup if required vars are missing.	Missing DATABASE_URL causes the app to crash on startup, not silently at query time.
In production (Render.com), vars are set in the dashboard, not in .env.	MEDSCAN uses Render environment groups so secrets never appear in source code.
python-dotenv loads a .env file in local development automatically.	.env.example documents required vars; the actual .env is in .gitignore.

MEDSCAN.AI: environment config

```
# MEDSCAN.AI -- config approach
import os
from dotenv import load_dotenv
load_dotenv()  # reads .env in local dev

DATABASE_URL = os.getenv('DATABASE_URL')   # Neon PostgreSQL
SECRET_KEY   = os.getenv('SECRET_KEY')     # JWT signing key
REDIS_URL    = os.getenv('REDIS_URL')      # Upstash Redis
S3_BUCKET    = os.getenv('SUPABASE_BUCKET')

# In production these come from Render environment variables
# -- never from a committed .env file
```

14. Async vs Sync Route Handlers

FastAPI supports both async def (non-blocking) and def (blocking, run in a thread pool) route handlers. Use async def when you await I/O operations (async DB drivers, async HTTP clients). Use def when you use synchronous libraries like SQLAlchemy (sync) or requests, since FastAPI runs them in a thread pool automatically to avoid blocking the event loop.

General FastAPI Concept	MEDSCAN.AI Usage
async def route(): await some_async_call() -- non-blocking I/O.	upload() uses async def because await file.read() is async (UploadFile).
def route(): -- FastAPI runs this in a thread pool, keeping the event loop free.	get_reports(), get_status() use def because SQLAlchemy is synchronous.
Do NOT use time.sleep() in async routes -- use asyncio.sleep() instead.	Celery tasks are separate processes, so sleeping there does not affect FastAPI.
Mixing sync SQLAlchemy with async FastAPI is fine -- FastAPI runs sync in threads.	MEDSCAN uses sync SQLAlchemy throughout; no async DB driver is needed.

MEDSCAN.AI: async vs sync

```
# async def: file read is awaitable (UploadFile is async)
@router.post('/upload')
async def upload_report(file: UploadFile = File(...), ...):
    contents = await file.read()  # non-blocking
    ...

# def: SQLAlchemy is sync, FastAPI runs in thread pool
@router.get('/reports')
def list_reports(db: Session = Depends(get_db), ...):
    return db.query(Report).filter(...).all()  # sync query
```

15. Calling the ML Microservice from Celery

MEDSCAN runs the ML pipeline (OCR + prediction) as a separate microservice on Render.com. The Celery task sends the uploaded file to the ML service via HTTP POST (multipart/form-data) and receives the prediction result as JSON. This decouples the API from the GPU-heavy ML workload.

General FastAPI Concept	MEDSCAN.AI Usage
The Celery task uses requests.post() to call the ML service URL.	process_medical_report sends the S3 file to ML_SERVICE_URL/process.
The ML service (FastAPI too) returns OCR text + risk predictions as JSON.	Response: {extracted_metrics: {...}, risks: {...}, risk_level: 'moderate'}.
ML_SERVICE_URL is set as an environment variable so it can differ per env.	Dev: http://localhost:8001, Production: the Render ML service URL.
Retries on network errors prevent a transient blip from failing the report.	Celery task retries up to 3 times with exponential back-off (2^retry seconds).

MEDSCAN.AI: ML microservice call

```
# task_queue/tasks.py -- calling the ML microservice
import requests

ML_SERVICE_URL = os.getenv('ML_SERVICE_URL')

@shared_task(bind=True, max_retries=3)
def process_medical_report(self, report_id, filename, report_type):
    # download file from S3
    s3 = boto3.client('s3', ...)
    obj = s3.get_object(Bucket=BUCKET, Key=filename)
    file_bytes = obj['Body'].read()

    # POST to ML microservice
    resp = requests.post(
        f'{ML_SERVICE_URL}/process',
        files={'file': (filename, file_bytes)},
        data={'report_type': report_type},
        timeout=120,
    )
    resp.raise_for_status()
    result = resp.json()

    # save to DB
    _save_ocr_results(report_id, result['extracted_metrics'])
    _save_prediction(report_id, result)
```

16. Startup and Shutdown Lifecycle Events

FastAPI lets you register functions that run when the app starts up and when it shuts down. This is useful for warming up caches, creating DB tables, loading ML models into memory, or closing connections cleanly.

General FastAPI Concept	MEDSCAN.AI Usage
@app.on_event('startup') runs once when the server first starts.	MEDSCAN creates DB tables (Base.metadata.create_all) on startup.
@app.on_event('shutdown') runs when the server is stopping.	Connection pools and caches can be cleaned up on shutdown.
In newer FastAPI, lifespan context manager replaces on_event (recommended).	MEDSCAN uses the older on_event style; lifespan is the future direction.
OCR model warm-up: loading PaddleOCR on first request adds latency.	get_ocr_runner() is called at startup to pre-load the OCR singleton.

MEDSCAN.AI: startup event

```
# MEDSCAN.AI -- app/main.py startup
from app.database import engine, Base

@app.on_event('startup')
def startup():
    Base.metadata.create_all(bind=engine)  # create tables if missing

# Modern lifespan alternative (FastAPI 0.93+):
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app):
    Base.metadata.create_all(bind=engine)  # startup
    yield
    # shutdown logic here

app = FastAPI(lifespan=lifespan)
```

17. MEDSCAN.AI Quick Route Reference

All routes in the MEDSCAN.AI API grouped by category.

Route	Description
POST /auth/register	Create new user account. Returns 201 + user object.
POST /auth/login	Authenticate user. Returns access_token + refresh_token.
POST /auth/refresh	Exchange refresh_token for a new access_token.
POST /api/upload	Upload medical report image/PDF. Returns report_id + 'processing'.
GET /api/reports	List all reports for the logged-in user.
GET /api/status/{report_id}	Poll processing status and retrieve prediction result.
DELETE /api/reports/{id}	Delete a report and its associated predictions.
POST /api/compare	Compare two reports. Returns trend data + delta risks.
GET /api/compare/{id}	Retrieve a previously run comparison result.
GET /api/history	Paginated history of all past reports with results.

18. End-to-End Request Lifecycle: GET /api/reports

Tracing a single request from browser to database and back illustrates how all the FastAPI concepts connect in practice.

1. Browser	React calls apiFetch('/api/reports') with Authorization: Bearer <token>.
2. CORS	CORSMiddleware checks the Origin header and adds Access-Control-Allow-Origin.
3. Router	FastAPI matches GET /api/reports to list_reports() in reports.py.
4. Auth	Depends(get_current_active_user) -> Depends(get_current_user) -> decodes JWT.
5. DB	Depends(get_db) opens a SQLAlchemy session for this request.
6. Handler	list_reports() queries db.query(Report).filter(user_id==...).all().
7. Serial.	FastAPI serializes the list of Report objects through ReportResponse schema.
8. Response	HTTP 200 JSON array is sent back; get_db() finally block closes the session.

Key takeaway: FastAPI, Depends(), SQLAlchemy, and Pydantic each handle one layer of the request. You write only the business logic in the handler; everything else is handled by the framework.